

Laboratorio di Informatica 1

Corso di Laurea in Fisica

Federico Nati, Uni. Milano-Bicocca, 2021

Laboratorio di Informatica 1

Docenti:

- Luca Bernardinello, Turno A-L
- Federico Nati, Turno M-Z

Lezioni frontali: 2 CFU, 16 Ore, 8 lezioni

Esercitazioni in lab: 2 CFU, 20 Ore, 9 esercitazioni (2 o 3 ore ciascuna)

Contatti:

Federico Nati
Tel. 02 6448 2324
Email: federico.nati@unimib.it
Stanza: 3054b, Ed. U1, Terzo Piano

Calendario del corso

- Date e orari di lezioni ed esercitazioni sul sito del corso;
- Faremo alcune lezioni di ripasso, domande, chiarimenti;
- Verso la fine del corso uno o più seminari di approfondimento;
- Prestare attenzione agli avvisi che invieremo tramite il sito.

Esercitazioni

- Divisione in quattro turni settimanali (A, B, C, D)
- Una esercitazione a settimana di 2h o 3h
- Due esercitatori
- Ogni esercitazione prevede una parte di spiegazione o introduzione, che integra concetti e nozioni spiegate a lezione e descrive il lavoro da svolgere
- In alcune date assegneremo esercizi da svolgere in gruppi di tre studenti e consegnare

Programma del corso

- **Obiettivi**
  - Nozioni fondamentali relative agli algoritmi, alla rappresentazione e all'elaborazione dei dati
  - Lettura e scrittura di programmi semplici, ma non banali, nel linguaggio C, che sviluppano soluzioni algoritmiche a problemi di ambito scientifico

- **Argomenti delle lezioni**
  - Nozioni di algoritmo e di costo computazionale di un algoritmo;
  - Struttura funzionale di un calcolatore. Natura e funzioni dei sistemi operativi;
  - Rappresentazione dei dati. Numerazione binaria. Rappresentazione dei numeri interi e dei numeri reali.
  - Rappresentazione a virgola mobile. Approssimazioni e errori;
  - Principi di programmazione nel linguaggio C
  - Variabili e tipi
  - Istruzioni primitive
  - Strutture di controllo del flusso (sequenza, scelta, iterazione)
  - Funzioni e passaggio di parametri
  - Tipi strutturati: vettori, matrici, record
  - Puntatori e allocazione dinamica della memoria
  - Modelli dei dati: liste concatenate, alberi, stack (pila), code, grafi;
  - Strategie di progettazione delle strutture dei dati e degli algoritmi.
  - Ordinamento dei dati
  - Ricorsione

Linguaggi di programmazione utilizzati

Notare anche gli incrementi

LI1

LI1, LF1 (cenni)

LI2

Jan 2021	Jan 2020	Change	Programming Language	Ratings	Change
1	2	↑	C	17.38%	+1.61%
2	1	↓	Java	11.96%	-4.93%
3	3		Python	11.72%	+2.01%
4	4		C++	7.56%	+1.99%
5	5		C#	3.95%	-1.40%
6	6		Visual Basic	3.84%	-1.44%
7	7		JavaScript	2.20%	-0.25%
8	8		PHP	1.99%	-0.41%
9	18	⇈	R	1.90%	+1.10%
10	23	⇈	Groovy	1.84%	+1.23%
11	15	⇈	Assembly language	1.64%	+0.76%
12	10	↓	SQL	1.61%	+0.10%

Fonte: TIOBE index, aggiornato mensilmente, e calcolato dal numero di risultati a seguito di ricerche con il nome del linguaggio attraverso motori di ricerca tra cui Google, Microsoft, Yahoo!, Wikipedia, YouTube

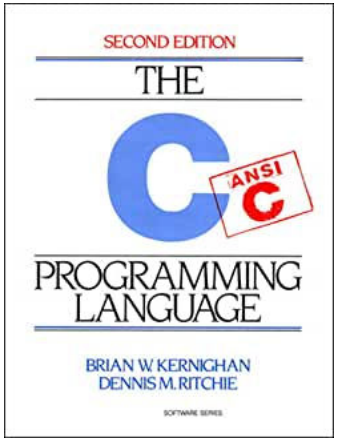
Testi consigliati

- J. G. **Brookshear** e S. G. **Kochan,** ***Fondamenti di informatica e programmazione in C***, Pearson, 2014. Comprende tutti gli argomenti trattati nel corso (e qualcosa di più). Alcuni capitoli sono disponibili solo in formato digitale (Nelle slide viene citato come **BK**);
- B.W. **Kernighan** e D.M. **Ritchie,** ***Il linguaggio C***, seconda edizione, Pearson-Prentice Hall, 2004. Scritto dai creatori del linguaggio C, viene considerato “la Bibbia” del C, ma non è un testo per iniziare, in quanto presuppone nel lettore almeno le conoscenze di base della programmazione (Nelle slide viene citato come **KR**);
- **L.M. Barone, E. Marinari, G. Organtini, F. Ricci-Tersenghi,** ***Programmazione scientifica***, Pearson Education, 2006, scritto da e per fisici, ricco di esempi scientifici (Nelle slide viene citato come **BMOR**).



Altri testi:

- Stephen G. Kochan, Programmare in C – Introduzione al linguaggio, Pearson, 2011. Destinato anche a principianti nella programmazione (una selezione dei capitoli di questo libro forma una parte del testo di Kochan e Brookshear);
- G.M. Schneider, J.L. Gersting, Informatica, Apogeo, 2007;
- Alfio Quarteroni, Fausto Saleri, Calcolo Scientifico, 4a edizione, Springer;
- B.W. Kernighan, R. Pike, The Practice of Programming, Addison-Wesley, 1999. Questo testo è destinato a lettori con qualche esperienza di programmazione, ma può essere utile anche a principianti. Ne esiste anche una traduzione in italiano.



Metodo di valutazione

- L'esame consiste in una prova scritta e in un colloquio orale;
- La prova scritta ha lo scopo di accertare la capacità di risolvere problemi attraverso la stesura e l'esecuzione di un programma in C;
- Il colloquio comprende una discussione della prova scritta e domande sul contenuto dell'insegnamento;
- I problemi consegnati dai gruppi alle esercitazioni potranno essere oggetto di discussione durante l'esame orale;
- Sul sito del corso metteremo (o sono già) a disposizione esempi di prove scritte assegnate in passato.

Coordinamento e propedeuticità con altri corsi

- Lab. Inf. 1 e Lab. Inf. 2
  - Le conoscenze e le competenze del LI1 sono necessarie al LI2
  - Nel corso di LI2 si introdurranno nuovi paradigmi di programmazione (programmazione a oggetti) e il linguaggio C++, che è un’estensione del C

- Lab. Inf. 1 e Laboratori di Fisica 1
  - La statistica e le tecniche di analisi dati insegnate nel LF2 potranno essere messe in pratica attraverso gli strumenti insegnati nel LI1
  - Forniremo alcuni esempi di utilizzo del C, del Python e altri strumenti a tale scopo. Tuttavia, essendo entrambi corsi del primo anno, sarà nel LI2 che tali tematiche verranno approfondite

Hardware e software necessari consigliati

- In mancanza di accesso ai laboratori, è necessario l'utilizzo un computer tipo PC o Mac, non importa quanto recente o potente. Qualora se ne fosse sprovvisti, comunicarlo via email.
- In questo corso si fa uso di sistemi operativi basati su GNU **Linux**, e di software open source con essi distribuiti.
- **Opzione 1**: Installare Linux in dual boot sul proprio computer. Consiglio la distribuzione Ubuntu, ma vanno bene anche altre;
- **Opzione 2**: VirtualBox con macchina virtuale Linux. Mettiamo a disposizione un'immagine da importare con la distribuzione Xubuntu;
- **Opzione 3**: Su Windows 10 esiste la possibilità di avere un sottosistema Linux;
- **Opzione 4**: Su Windows, installare Cygwin e gcc;
- **Opzione 5**: Su MacOS si può usare il terminale testuale, ma va installato gcc;
- **Opzione 6**: Mini computer Raspberry PI (versione 3 o 4), da collegare a un monitor esterno. Costo contenuto, utilizzabile con con sistema operativo Linux (Raspberry PI OS o Ubuntu);
- **Opzione 7**: Usare la macchina virtuale Linux del laboratorio.
- **Opzione 8**: Usare una distribuzione di tipo “Live”, cioè che si installa e usa da chiavetta USB (es. Q4OS)

- Editor di testo: Vim (guida sul sito), GVim, GEdit, Notepad++, Atom, Emacs, Sublime Text

- Rispondere al sondaggio sulla dotazione personale che si trova sul sito del corso.

Partiamo dal nome del corso:

Laboratorio di Informatica 1

- Perché **Laboratorio**:
  - Si mettono in pratica concetti e nozioni acquisite nel corso attraverso l’esperienza diretta;
  - Si impara a utilizzare apparecchiature e tecnologie per risolvere problemi di natura scientifica;
  - Salvo restrizioni dovute alla pandemia, l’università fornisce dei laboratori attrezzati per le esperienze.
- Cosa è l’**Informatica**?
- Perché **1?** Perché, ovviamente, è il primo e del primo anno, quindi cercheremo di non dare per scontato nulla e di chiarire i concetti a partire dall'inizio. Molti di questi concetti verranno però solo accennati o delineati, quindi potreste trovarvi a doverli approfondire in futuro.

Cosa è l'Informatica

È una disciplina estremamente vasta che studia

- Gli algoritmi
- La rappresentazione dei dati
- L'informazione
- Teoria del calcolo, logica, matematica
- Cosa può essere automatizzato
- L'automatizzazione del calcolo
- Sistemi elettronici automatizzati
- Teoria e applicazione dei linguaggi di programmazione

Questi sono gli aspetti essenziali

"Dobbiamo disfarci del mito che l'informatica sia la scienza dei computer. L'informatica non riguarda i computer più di quanto l'astronomia non riguardi i telescopi, la biologia non riguardi i microscopi, o la chimica non riguardi ampolle e provette."

Michael R. Fellows, Ian Parberry (1993)

L’informatica e le altre scienze

Si accompagna e si integra o è di supporto a tutte le discipline scientifiche, e praticamente a tutte le discipline della conoscenza, dalla biotecnologia alla fisica, dall'economia alla storia, dalla statistica alla medicina.

L'informatica è fondamentale per la fisica moderna, e la fisica è importante per l'informatica a sua volta.

Il metodo sperimentale stesso è evoluto grazie alla disponibilità di strumenti di calcolo un tempo impensabili. La costruzione della conoscenza scientifica passa attraverso la realizzazione di simulazioni al computer e il software per l'analisi di enormi quantità di dati.

Nella vostra carriera di fisici potrete affrontare ideazione di algoritmi, scrittura di codici per la loro messa in pratica, realizzazione di software per simulazione di processi fisici, per analisi e visualizzazione di dati sperimentali, ingegnerizzazione di codici per il controllo di strumenti ed esperimenti scientifici, sviluppo di modelli statistici per via computazionale.

Nota terminologica

- **Hardware** si riferisce ai componenti fisici di una macchina e indica tutto ciò che è tangibile, come parti elettroniche, elettriche, meccaniche, magnetiche, eccetera.

- **Software** in generale indica tutto ciò che non è tangibile, per esempio i dati, le informazioni, i programmi, le sue istruzioni. Il sofware può essere scritto, salvato, immagazzinato in diversi tipi di supporto fisico;

- Tuttavia a volte si usa la parola **software come sinonimo di programma, applicazione, applicativo**, per questo sentirete che può “girare” (in inglese: run) su un computer, può essere caricato nella memoria di un calcolatore, può essere copiato o cancellato.

- **Computer e calcolatore** sono sinonimi (Attenzione: ***compilatore NON significa calcolatore***, si riferisce a preciso tipo di programma, lo vedremo).

- Sentirete spesso il termine inglese alternarsi a quello italiano per riferirsi alla stessa cosa.

- Inoltre, ma cerchermo di evitarlo, per colpa delle **cattive abitudini**, potrà succedere di sentire italianizzazioni di termini inglesi nel gergo informatico (es. **implement**are, **boot**are, ecc.).

Cosa è “informazione”

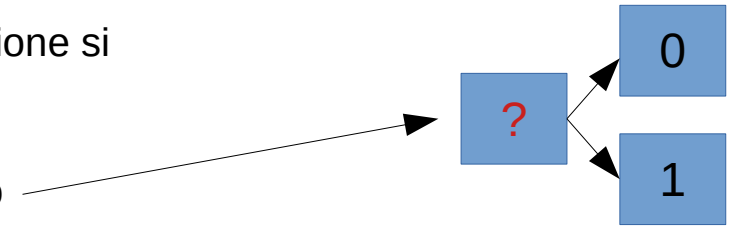
- Accenniamo soltanto, per evitare ambiguità alla teoria dell’informazione
- La teoria dell’informazione è una disciplina dell'informatica e delle telecomunicazioni che si occupa di misurazione, trasmissione, analisi ed elaborazione di informazioni;
- Si definisce “**bit**” l’unità fondamentale di informazione, e rappresenta quantità minima di informazione che serve a discernere tra due eventi equiprobabili;
- La grandezza che misura la quantità di dati prende il nome di entropia ed è solitamente espressa come numero di bit necessari per immagazzinare o trasmettere l'informazione;
- "A Mathematical Theory of Communication" Claude E. Shannon, 1948

Bit e informazione

[Si veda per esempio BK, Cap. 1.5]

In contesti informatici non legati alla teoria dell'informazione si parla di bit termini leggermente diversi

- Un bit è una quantità minima di informazione che può assumere solo due stati;
- I due stati sono interpretabili in diversi modi, per esempio" Vero o Falso, Bianco o Nero, Positivo o Negativo, Acceso o Spento, Sì o No, eccetera);
- Una **cifra in base 2** in informatica viene chiamata "bit" (Binary digIT). Un singolo bit come cifra binaria può valere o zero o uno;
- Una sequenza di N bit, cioè un numero binario a N cifre, può assumere 2^N diversi valori, corrispondenti a diverse combinazioni delle singole cifre binarie.



1 bit	2 bits	3 bits	4 bits	
0	00	000	0000	1000
1	01	001	0001	1001
	10	010	0010	1010
	11	011	0011	1011
		100	0100	1100
		101	0101	1101
		110	0110	1110
		111	0111	1111

Bit, Byte, rappresentazione dati

Imparare a memoria:

- 2^n con n = [0, 16]

E gli ordini di grandezza (in potenze di 10) di

- 2^{24} ~ 16 milioni
- 2^{32} ~ 4 miliardi
- 2^{64} ~ $1.9 \cdot 10^{19}$

N	2**N
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	**1024**
11	2048
12	4096
13	8192
14	16384
15	32768
16	**65536**

Sistemi binario ed esadecimale

Base 10:

1450.23 =

$1 \cdot 1000 + 4 \cdot 100 + 5 \cdot 10 + 0 \cdot 1 + 2 \cdot 0.1 + 3 \cdot 0.01 =$

$1 \cdot 10^{3} + 4 \cdot 10^{2} + 5 \cdot 10^{1} + 0 \cdot 10^{0} + 2 \cdot 10^{-1} + 3 \cdot 10^{-2}$

Base 10
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Sistemi binario ed esadecimale

Base 10:

$1450.23 =$

$1 \cdot 1000 + 4 \cdot 100 + 5 \cdot 10 + 0 \cdot 1 + 2 \cdot 0.1 + 3 \cdot 0.01 =$

$1 \cdot 10^3 + 4 \cdot 10^2 + 5 \cdot 10^1 + 0 \cdot 10^0 + 2 \cdot 10^{-1} + 3 \cdot 10^{-2}$

Base 2:

$1010.11 =$

$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} =$

$1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 + 1 \cdot 1/2 + 1 \cdot 1/4 =$

$10 + ¾ = 10.75$

Base 10	Binario
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Sistemi binario ed esadecimale

Base 10:

1450.23 =

$1 \cdot 1000 + 4 \cdot 100 + 5 \cdot 10 + 0 \cdot 1 + 2 \cdot 0.1 + 3 \cdot 0.01 =$

$1 \cdot 10^3 + 4 \cdot 10^2 + 5 \cdot 10^1 + 0 \cdot 10^0 + 2 \cdot 10^{-1} + 3 \cdot 10^{-2}$

Base 2:

1010.11 =

$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} =$

$1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 + 1 \cdot 1/2 + 1 \cdot 1/4 =$

10 + ¾ = 10.75

Base 16:

1A2F =

$1 \cdot 16^3 + A \cdot 16^2 + 2 \cdot 16^1 + F \cdot 16^0 =$

$1 \cdot 4096 + A \cdot 256 + 2 \cdot 16 + F \cdot 1 =$

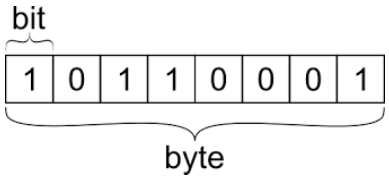
$1 \cdot 4096 + 10 \cdot 256 + 2 \cdot 16 + 15 \cdot 1 = 6703$

Base 10	Binario	Esadecimale
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	00010000	10
17	00010001	11
18	00010010	12
19	00010011	13
20	00010100	14
21	00010101	15
22	00010110	16
23	00010111	17
24	00011000	18
25	00011001	19
26	00011010	1A
27	00011011	1B
28	00011100	1C
29	00011101	1D
30	00011110	1E
31	00011111	1F

Bit e Byte

- Un numero a 4 bit può assumere $2^4 = 16$ combinazioni diverse di bit. Ovvero, con 4 bit posso immagazzinare informazioni relative a quantità che possono assumere fino a 16 valori diversi (es. 16 gradazioni di grigio, 16 diversi colori, 16 livelli di tensione che può assumere un segnale, una delle cifre in base 16, cioè una cifra **esadecimale**, eccetera)

- **Una seguenza di 8 bit si dice byte**. Un byte può assumere $2^8 = 256$ valori diversi



- **Un byte** si può scrivere usando **due cifre esadecimali,** es.

  `0001 1010 = 11010 = 1A`

- **Nota:** Per distinguere le basi si usa scrivere:

  `0b``11010 = `**`0x`**`1A`

  oppure: `11010`**`B`**` = 1A`**`H`**` = 26`**`D`**

  `(dove B sta per Binary, H per Hexadecimal, D per decimal)`

Base 10	Binario	Esadecimale
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	00010000	10
17	00010001	11
18	00010010	12
19	00010011	13
20	00010100	14
21	00010101	15
22	00010110	16
23	00010111	17
24	00011000	18
25	00011001	19
26	00011010	1A
27	00011011	1B
28	00011100	1C
29	00011101	1D
30	00011110	1E
31	00011111	1F

Immagazzinare informazione binaria

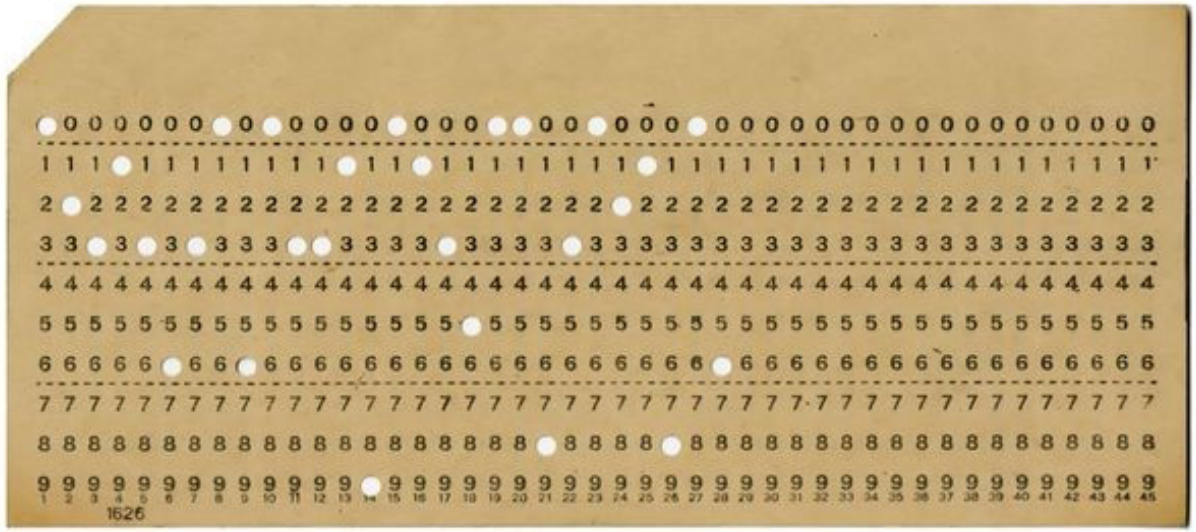
Nel gioco della tombola vogliamo immagazzinare sulla cartella di gioco un’informazione che può assumere solo due possibili stati: ESTRATTO o NON ESTRATTO. Non importa con che cosa decidiamo di coprire il numero, se con un fagiolo, un pezzo di buccia di mandarino, o altro. L’unica informazione rilevante è se il numero è coperto o meno.

Il fatto che siano solo DUE gli stati, ci facilita la vita.



Immagazzinare informazione binaria

Schede perforate





Univac 9200/II, 32KB di memoria, 1966

Funzionante (!) al museo Technikum 29 di Francoforte

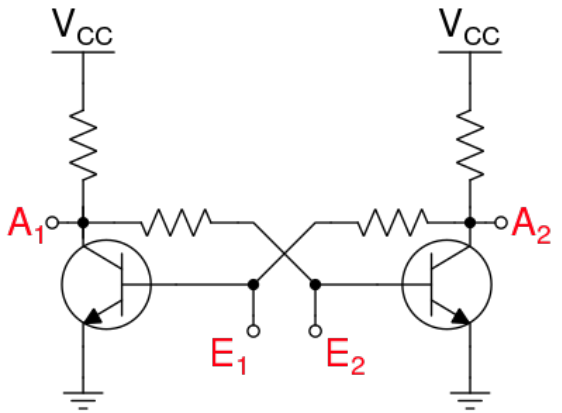
Circuiti elettronici

Un circuito elettronico digitale si basa su un numero finito di livelli di tensione elettrica.

Alcuni dispositivi sono in grado di cambiare e/o mantenere una certa tensione in uscita in funzione della tensione in ingresso.

Data una soglia per le tensioni elettriche, si può facilmente distinguere (anche in presenza di rumore e con una certa tolleranza) tra due stati per la tensione immagazzinata identificati con l'**uno** o con lo **zero.**

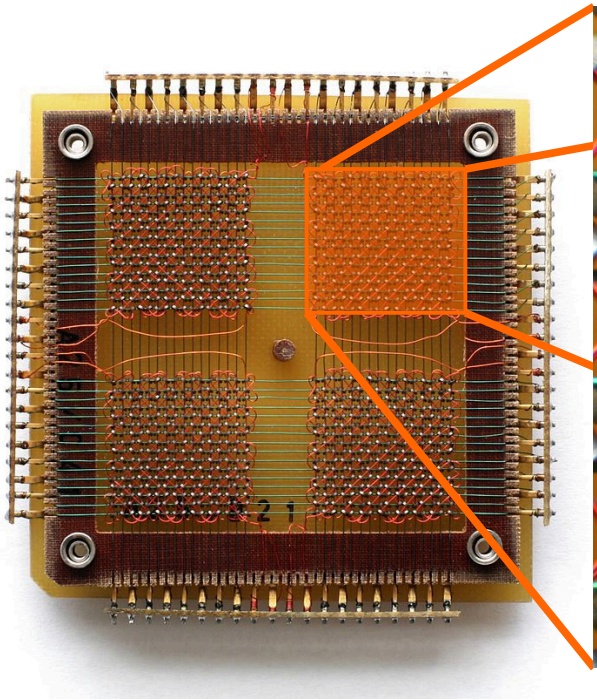
I due livelli di tensione usati nei circuiti digitali rappresentano i numeri binari 0 e 1, detti livelli logici.

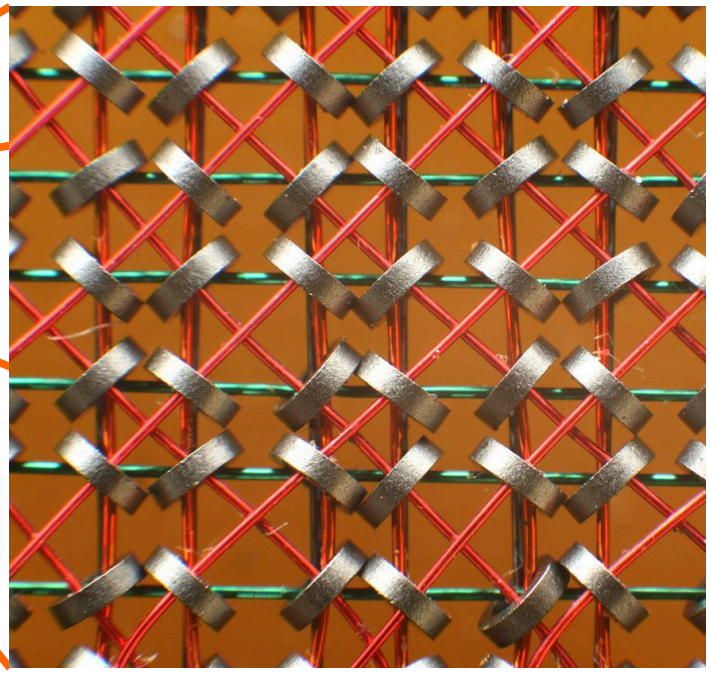


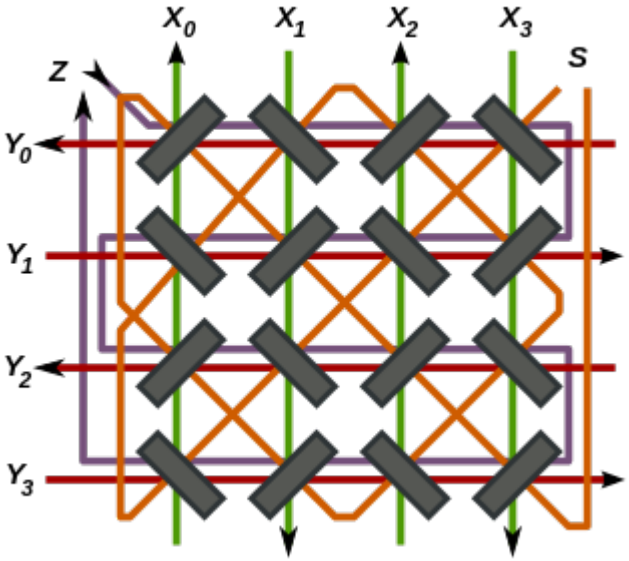
Un possibile circuito flip-flop



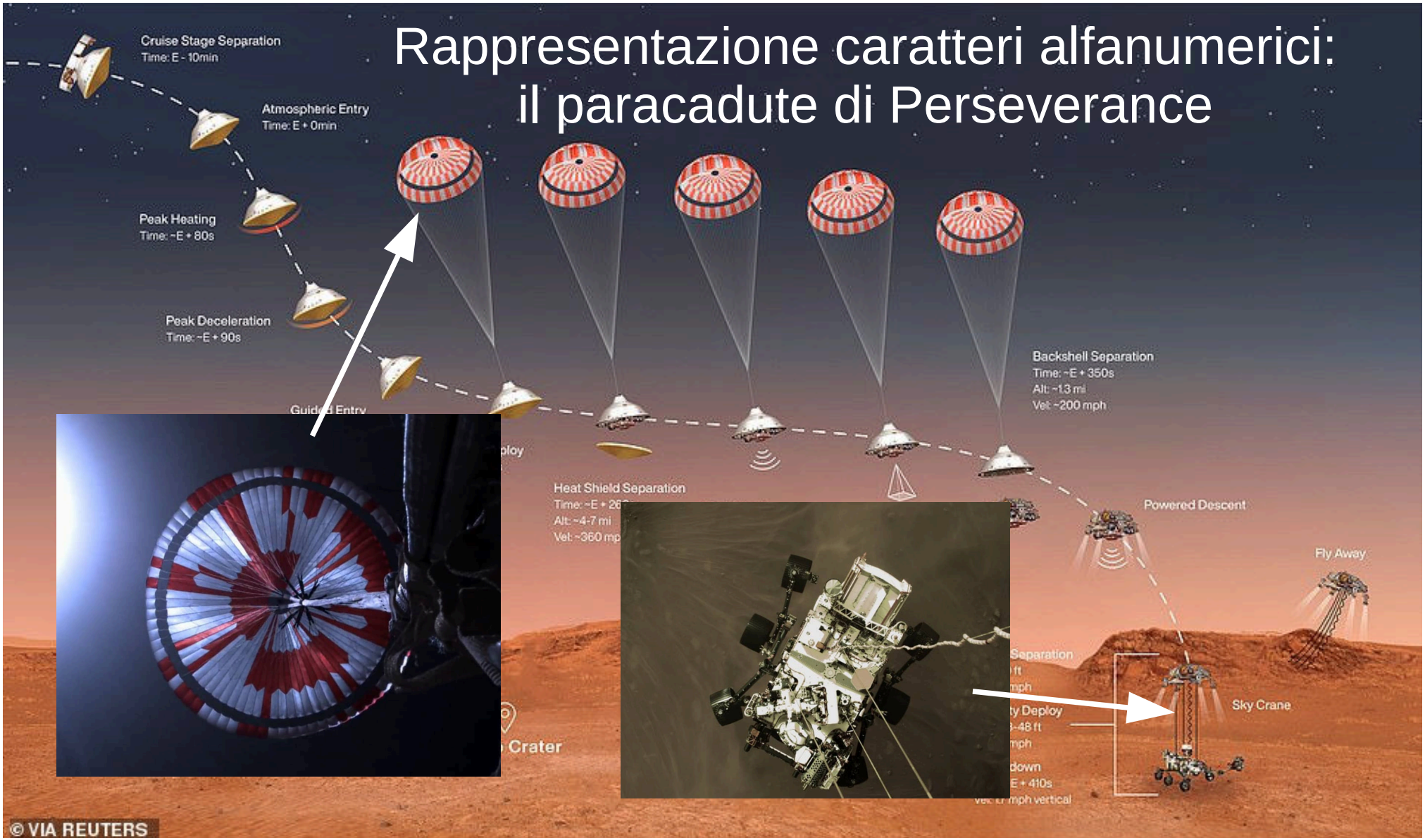
Memorie a nuclei magnetici



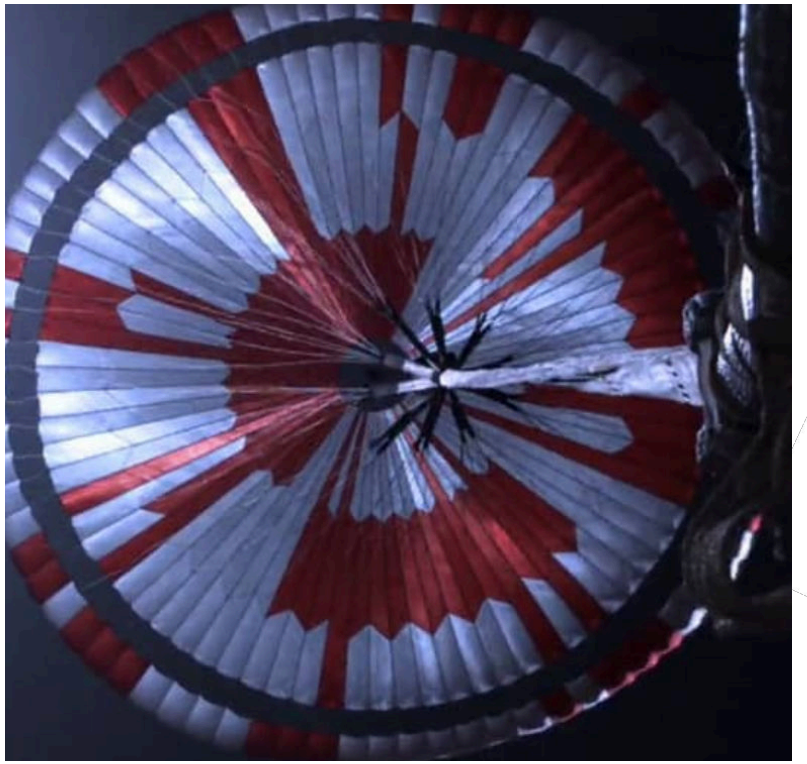


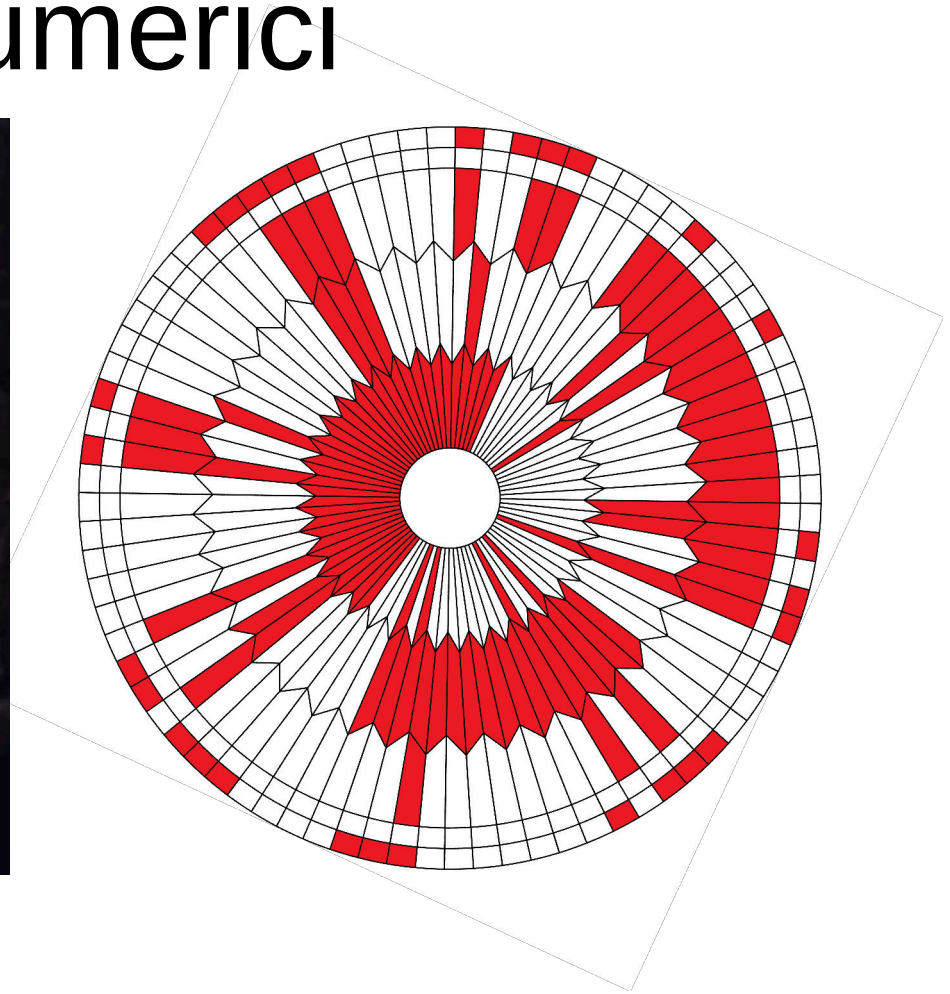


Rappresentazione caratteri alfanumerici: il paracadute di Perseverance

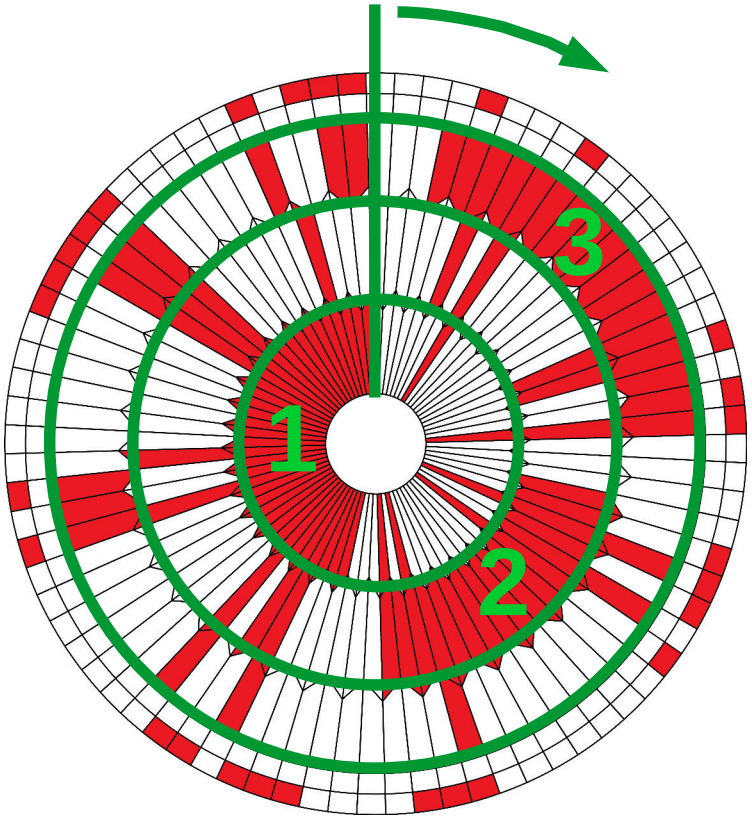


Rappresentazione caratteri alfanumerici





Rappresentazione caratteri alfanumerici



ROSSO=1
BIANCO=0

1

```
0000000100 4        D
0000000001 1        A
0000010010 18       R
0000000101 5        E
0001111111
1111111111
1111111111
1111111111
```

2

```
0000010100 20       T
0000011001 25       Y
0001111111
1111111111
0000001101 13       M
0000001001 9        I
0000000111 7        G
0000001000 8        H
```

3

```
0001111111
1111111111
0000010100 20       T
0000001000 8        H
0000001001 9        I
0000001110 14       N
0000000111 7        G
0000010011 19       S
```





Motto nel laboratorio del JPL, le cui coordinate terrestri sono appunto 34°11’58” N 118°10’31” W

Bit e “dinamica”

Una sequenza di N bit può rappresentare 2^N stati diversi

Il numero di bit esprime la “dinamica” con cui può essere rappresentata una grandezza fisica, oppure quanti diversi valori può assumere una certa informazione

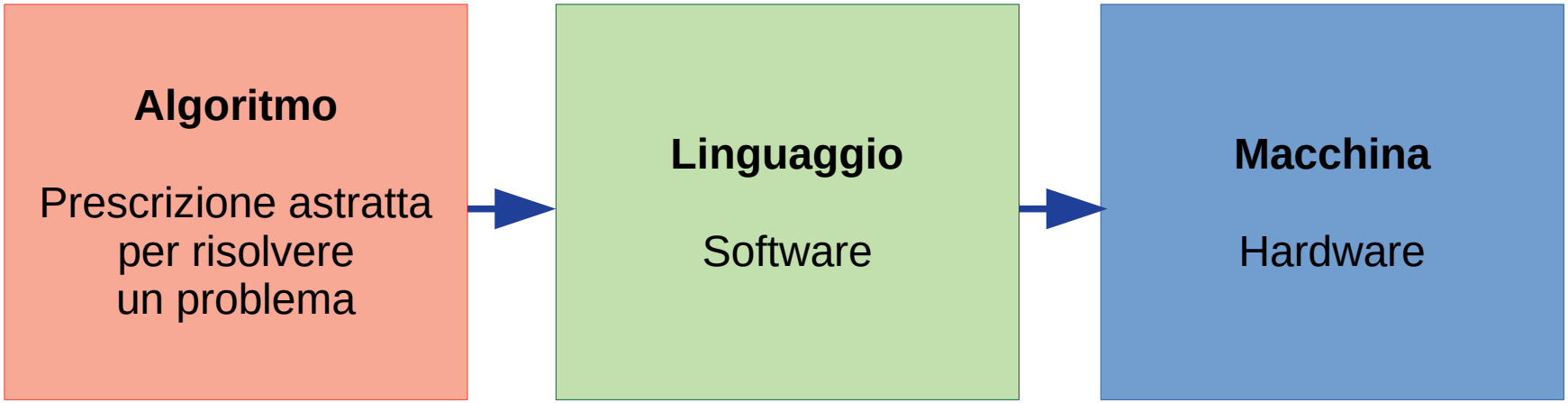
Si può costruire una tabella che metta in relazione diverse combinazioni di bit con diversi valori di una certa quantità. Dato un insieme di M diversi elementi, ho bisogno di una dinamica di minimo N bit tale che $2^N > M$

Per esempio, date **26** lettere diverse, posso costruire una tabella che le metta in relazione con sequenze di **5 bit**, in quanto $2^5 = 32 > 24$. Se usassi 4 bit potrei costruire solo 16 diverse combinazioni.

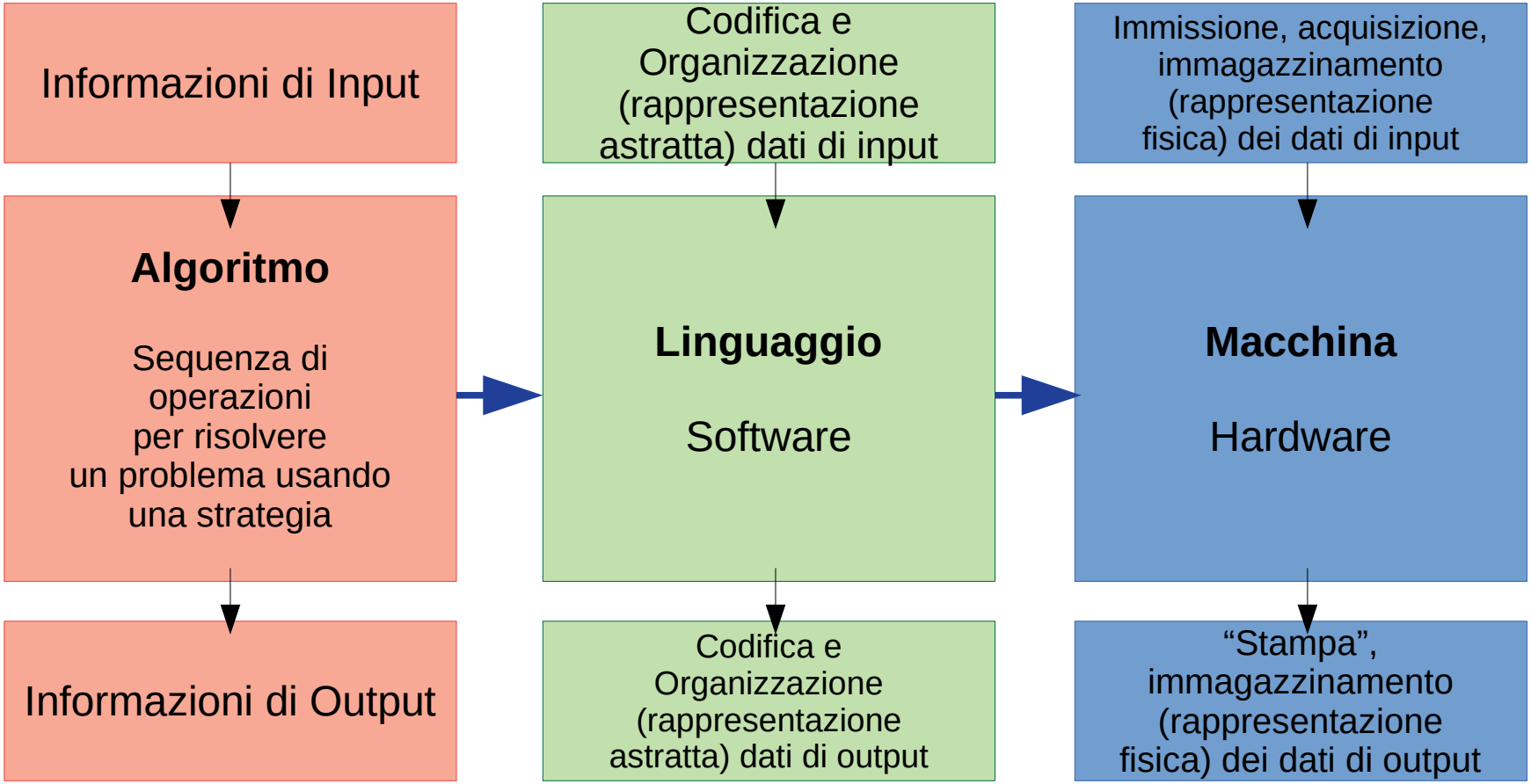
Più avanti vedremo maggiori dettagli su come effettivamente vengono rappresentati numeri, caratteri e altri tipi di dato nei calcolatori.

		5 BIT	4 BIT
1	A	00000	0000
2	B	00001	0001
3	C	00010	0010
4	D	00011	0011
5	E	00100	0100
6	F	00101	0101
7	G	00110	0110
8	H	00111	0111
9	I	01000	1000
10	J	01001	1001
11	K	01010	1010
12	L	01011	1011
13	M	01100	1100
14	N	01101	1101
15	O	01110	1110
16	P	01111	1111
17	Q	10000	
18	R	10001	
19	S	10010	
20	T	10011	
21	U	10100	
22	V	10101	
23	W	10110	
24	X	10111	
25	Y	11000	
26	Z	11001	
27		11010	
28		11011	
29		11100	
30		11110	
31		11111	

Agoritmi, linguaggi, macchine

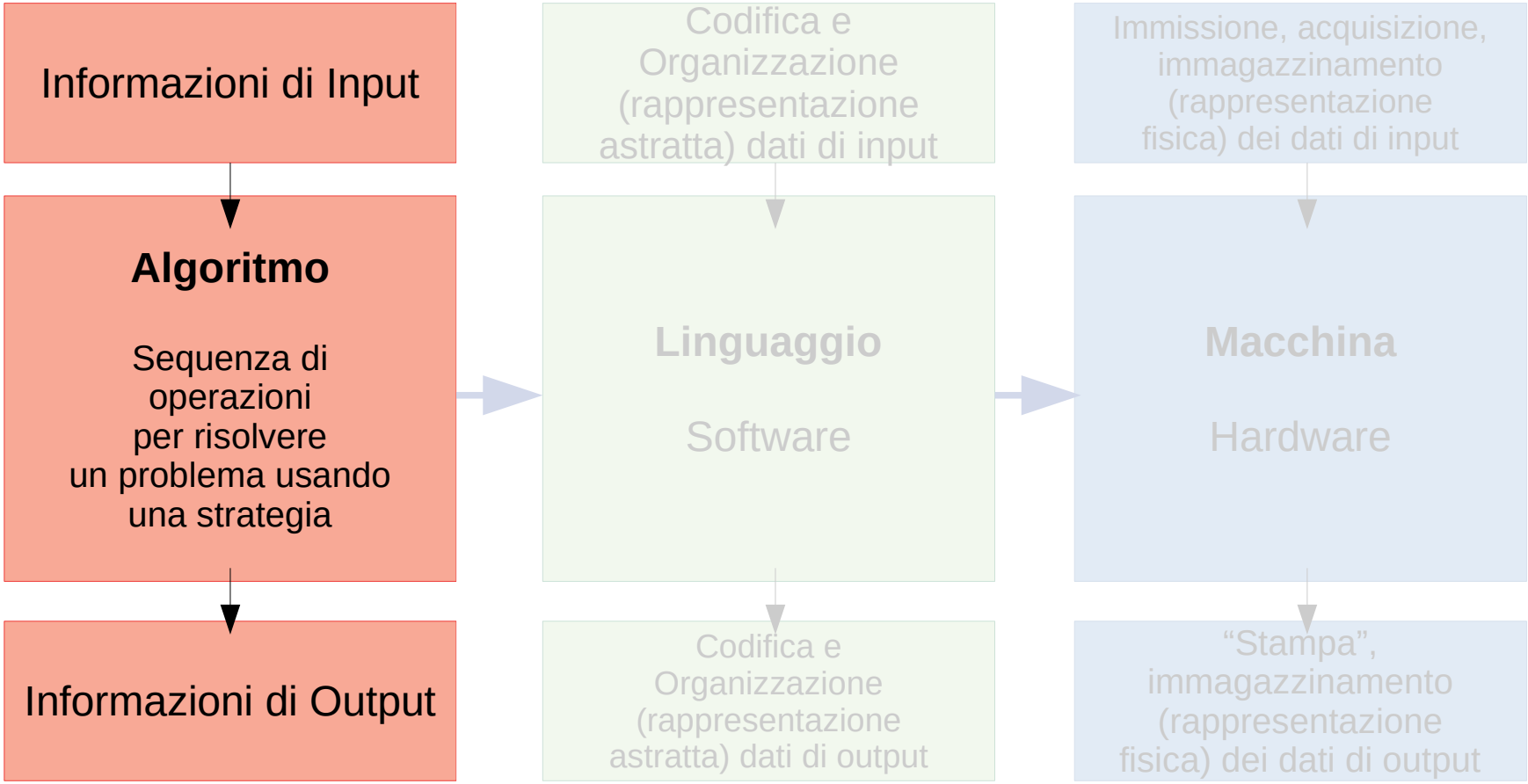


Dato un problema...



…otteniamo la soluzione

Dato un problema...



…otteniamo la soluzione

Cosa è un algoritmo

Un algoritmo è una sequenza finita di istruzioni ben determinate ed eseguibili attraverso un procedimento meccanico, cioè un procedimento realizzabile da una macchina priva di libero arbitrio, che produce con certezza una soluzione per una classe di problemi.

Deve essere

- **Finito**: costituito da un numero finito di istruzioni, una sequenza che ha una fine;
- **Deterministico**: dai dati in input, si ottengono i medesimi risultati in output;
- **Non ambiguo**: indicazioni chiare, eseguibili nello stesso modo indipendentemente dall'esecutore;
- **Generale**: offre una soluzione per tutti i problemi della stessa classe.

- Il termine deriva dal nome del matematico persiano al-Khwarizmi (IX secolo d.C.), uno dei primi ad aver fatto riferimento a questo concetto nel libro “Regole di ripristino e riduzione”.

- Un problema è calcolabile quando è risolvibile mediante un algoritmo

- [Si veda per esempio BK, Cap 3]

Esempi di algoritmi

- Una ricetta culinaria
- Le istruzioni per realizzare un origami
- Metodi matematici come il metodo di Ruffini, il metodo di Eulero, le operazioni in colonna, la formula per le soluzioni di un'equazione di secondo grado
- Le soluzioni a problemi geometriche per costruzioni grafiche, come il tracciamento della bisettrice di un angolo
- Un protocollo medico
- Delle indicazioni stradali per andare da un posto a un altro
- Si sente spesso parlare degli *algoritmi* dei social media o dei motori di ricerca

Cosa sono i dati

- I dati del problema sono rappresentazioni delle informazioni in ingresso (INPUT), in uscita (OUTPUT), e di qualsiasi informazione viene temporaneamente immagazzinata durante l'esecuzione dell'algoritmo;
- I dati rappresentano informazioni relative a quantità su cui è possibile eseguire operazioni logico-matematiche;
- I dati possono essere di diverso **tipo**: il tipo è un attributo che ci dice come **interpretare** e come **codificare** la rappresentazione dell'informazione;
- Il tipo dei dati definisce anche le **operazioni che si possono fare con i dati;**
- I tipi possono essere i più disparati, per esempio interi, decimali, reali, positivi, negativi, lettere, testi, immagini, suoni, simboli, ma tutti i tipi possono essere rappresentati da numeri o sequenze di numeri;
- I dati possono essere organizzati in **strutture** di dati che ne definiscono le reciproche relazioni.

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

Individuo i dati del problema:

- **N**, dato di INPUT
- **i**, variabile che può assumere valori interi da 1 a N
- **i^2**, di volta in volta il termine della somma
- **SOMMA**, il risultato dell'accumulazione dei valori di i^2
- L'ultimo valore assunto da SOMMA sarà il risultato che si vuole ottenere, cioè il dato di OUTPUT, cioè il valore di f(N)

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		

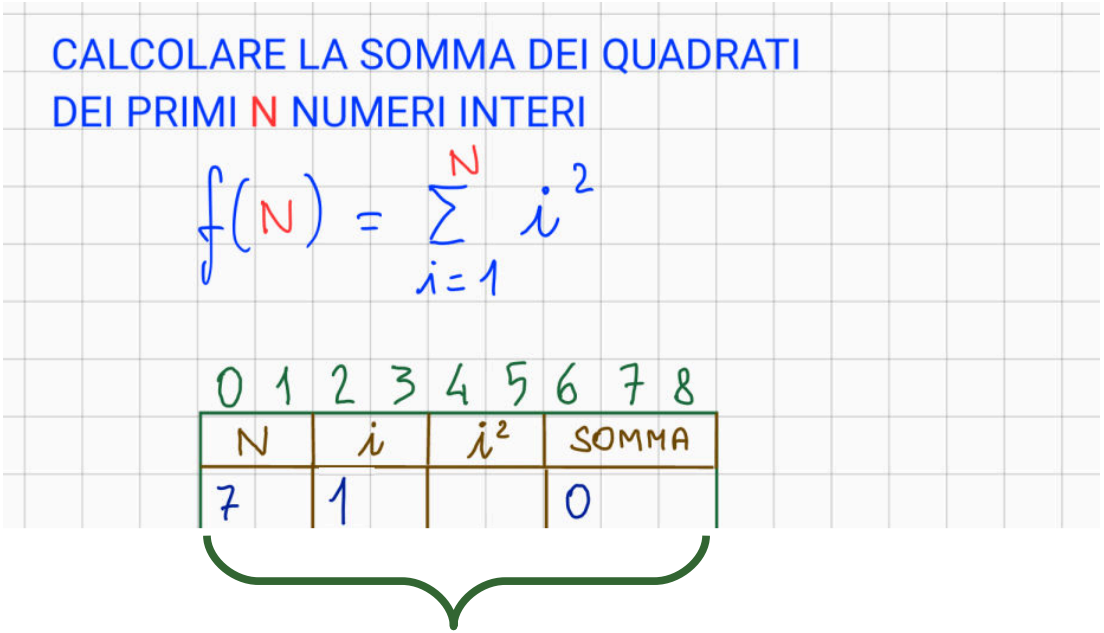
- Riserva una riga di 9 quadretti, di cui 2 per N, 2 per i, 2 per i^2, tre per SOMMA

CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		
7		1				0		

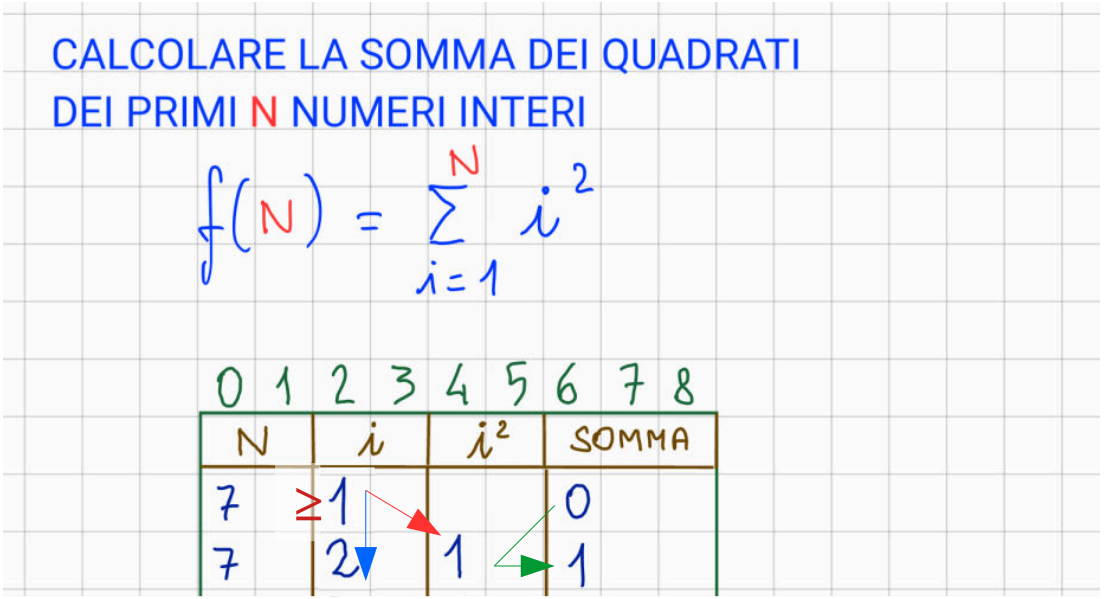
- Riserva una riga di 9 quadretti, di cui 2 per **N**, 2 per **i**, 2 per **i²**, tre per **SOMMA** (una cifra per ogni quadretto)
- Leggi **N** da input e scrivilo nella sua *cella*
- Scrivi 1 nella cella chiamata **i**
- Scrivi 0 nella cella chiamata SOMMA



Questi 9 quadretti sono l'analogo dell'occupazione di memoria necessario alla rappresentazione dei dati

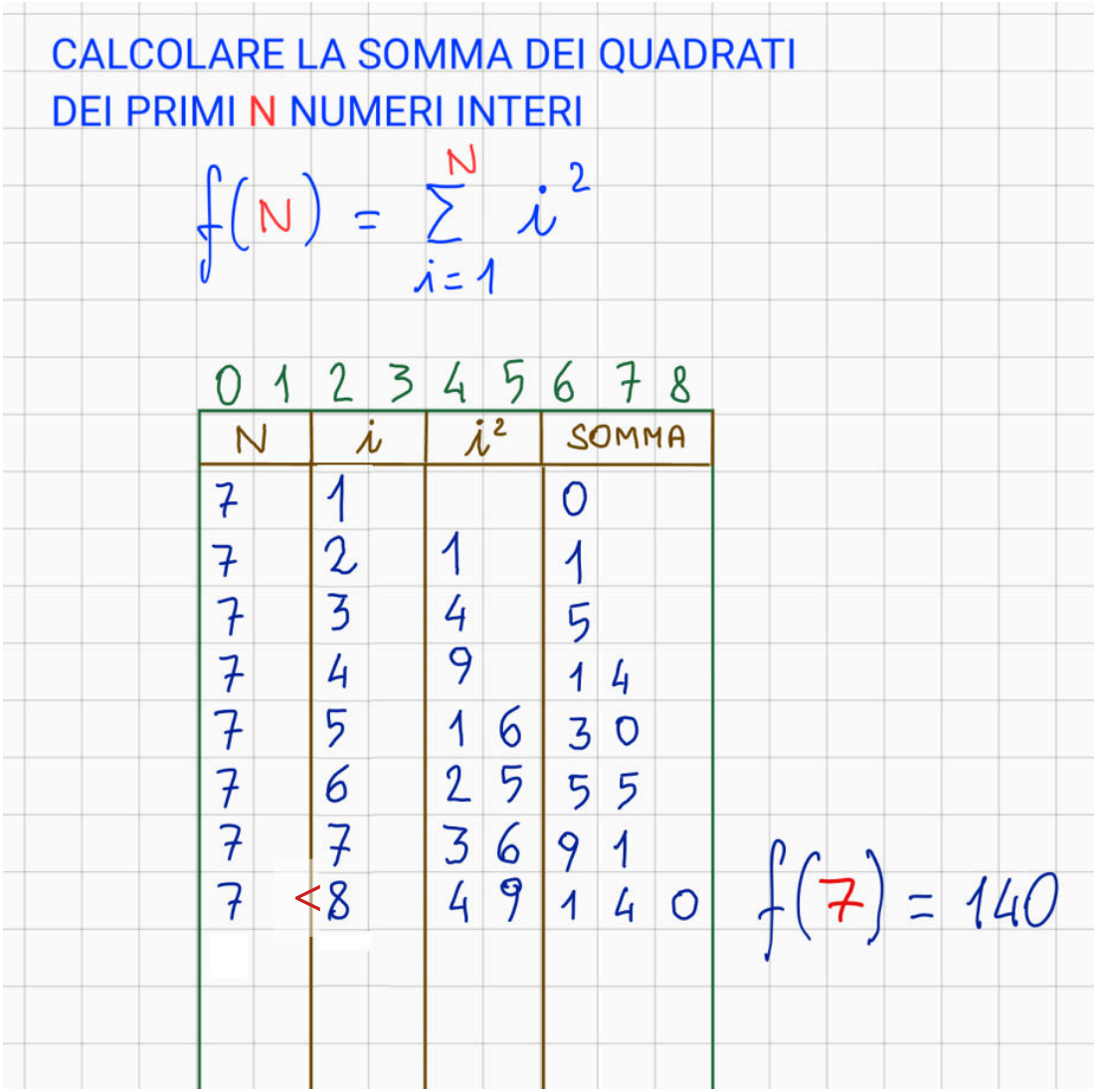
Ogni *cella:*

- è costituita da uno più quadretti
- Ha un numero identificativo, analogo all'**indirizzo**
- Contiene un valore, in questo caso una rappresentazione di numeri interi con cifre decimali



... segue dalle istruzioni di prima:

- Controlla se i è minore uguale a N.
  - Se sì, allora:
    - Calcola i^2 e scrivilo nella sua cella
    - Aggiungi il valore di i^2 al valore di SOMMA, e scrivilo nella cella corrispondente
    - Aumenta il valore di i di una unità e scrivilo nella sua cella



... segue dalle istruzioni di prima:

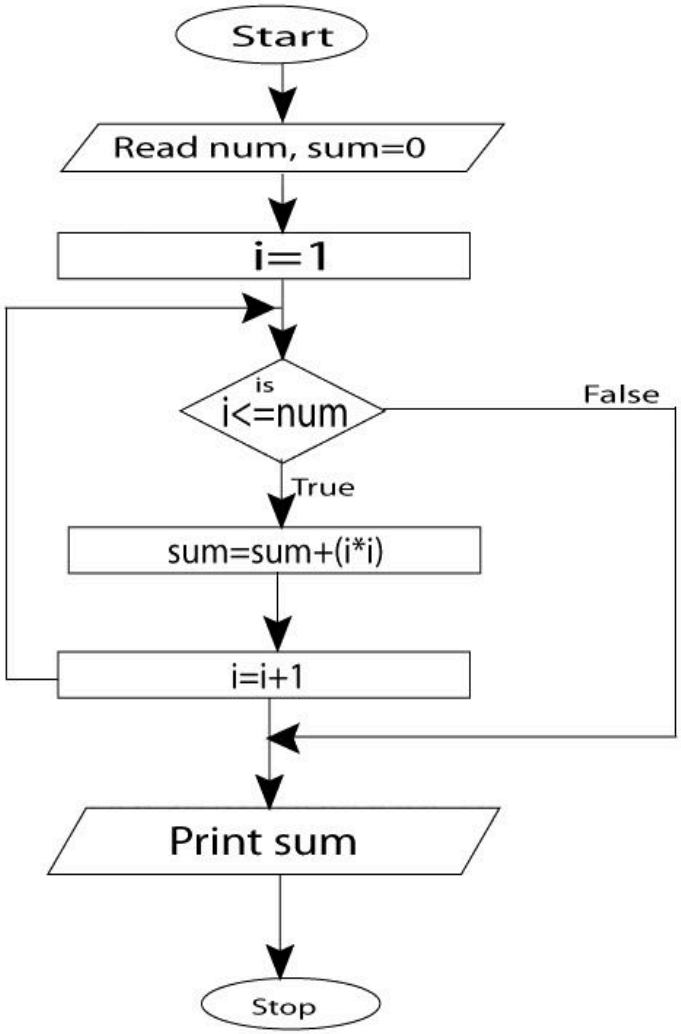
- Controlla se i è minore uguale a N.
  - Se sì, allora:
    - Calcola i^2 e scrivilo nella sua cella
    - Aggiungi il valore di i^2 al valore di SOMMA, e scrivilo nella cella corrispondente
    - Aumenta il valore di i di una unità e scrivilo nella sua cella
    - Ripeti
  - Se no, comunica il valore di SOMMA, quindi termina le operazioni

CALCOLARE LA SOMMA DEI QUADRATI DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0 N	1 2 i	3 4 i^2	5 6 7 8 SOMMA
7	1		0
7	2	1	1
7	3	4	5
7	4	9	14
7	5	16	30
7	6	25	55
7	7	36	91
7	8	49	140

$f(7) = 140$



CALCOLARE LA SOMMA DEI QUADRATI
DEI PRIMI N NUMERI INTERI

$$f(N) = \sum_{i=1}^{N} i^2$$

0	1	2	3	4	5	6	7	8
N		i		i^2		SOMMA		
7		1				0		
7		2		1		1		
7		3		4		5		
7		4		9		1	4	
7		5		1	6	3	0	
7		6		2	5	5	5	
7		7		3	6	9	1	
7		8		4	9	1	4	0

0.25 €
0.25 €
0.25 €
0.25 €
0.25 €
0.25 €
0.25 €
0.25 €

Tot: 1.75 €

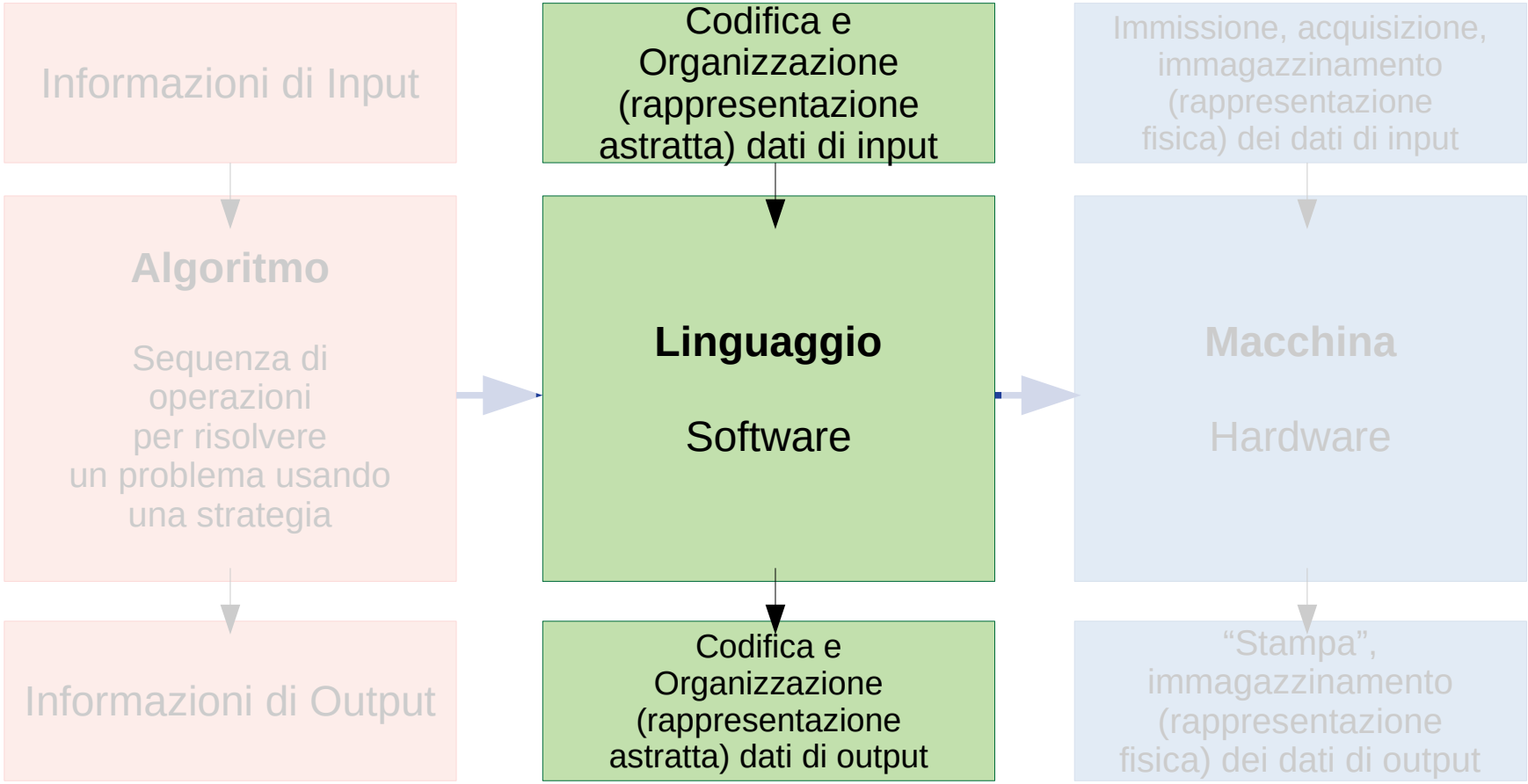
Un primo accenno al concetto di **COSTO COMPUTAZIONALE** di un algoritmo. Immaginiamo per gioco che le nostre istruzioni le esegua un addetto che chiede un *compenso monetario* di 0.25 € ogni volta che calcola un elevamento al quadrato (quindi *non in funzione del tempo* che impiega).

Il prezzo pagato alla fine, a parità di N, potrebbe aumentare o ridursi a seconda del metodo, della strategia, dell'algoritmo utilizzato.

Di un algoritmo che riduce mediamente il numero di istruzioni da eseguire rispetto a uno che richiede più passaggi **a parità di dati di ingresso**, si dice che ha un costo computazionale inferiore.

Naturalmente questo esempio monetario è solo un gioco, il costo di un algoritmo non è un costo in denaro. Sebbene...

Dato un problema...



…otteniamo la soluzione

Cosa è un programma

- Il termine programma può riferirsi a tre significati distinti: codice sorgente, file eseguibile, processo in esecuzione. Vediamo innanzitutto il primo dei tre.

- Un programma inteso come codice sorgente è un testo espresso in un linguaggio di programmazione;

- Dato un problema da automatizzare, la programmazione traduce l'algoritmo nel codice sorgente di un programma, scritto in un certo linguaggio;

- Un programma inteso come file eseguibile è la traduzione del codice sorgente in un linguaggio comprensibile dalla macchina che lo deve eseguire.

- Macchine in grado di essere programmate, i computer o calcolatori, possono quindi eseguire il programma, riflettendone la logica di elaborazione, e fornire la soluzione al problema. Quando il programma è in esecuzione dalla macchina, ci si riferisce ad esso con il termine processo.

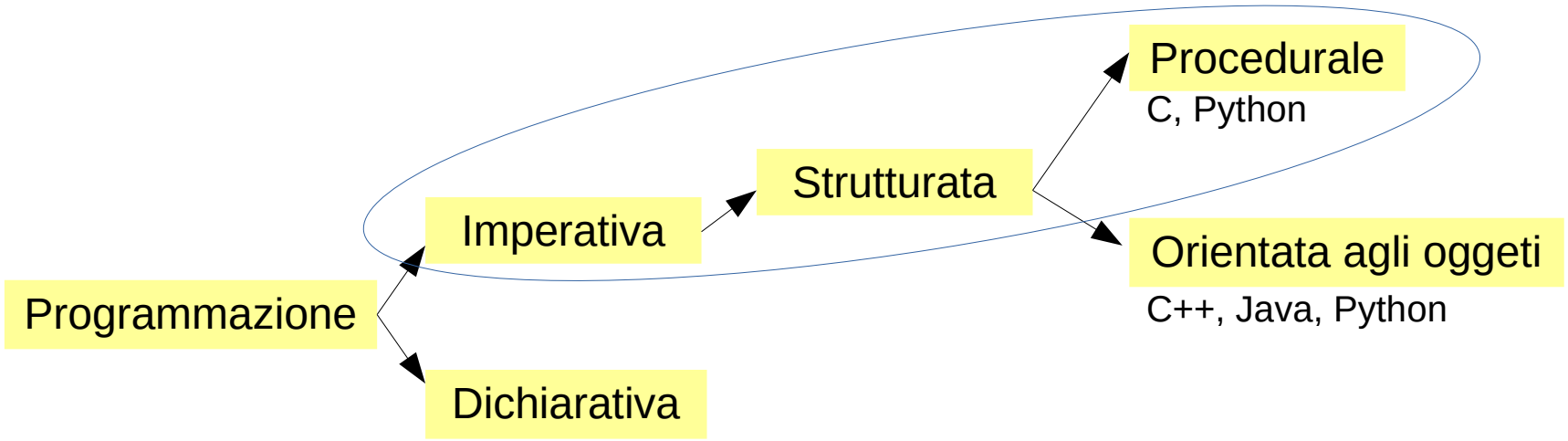
Cosa è un linguaggio di programmazione

- I linguaggi, in generale, si dividono in linguaggi formali e naturali;
- I linguaggi naturali sono quelli parlati dalle persone. Sebbene gli esseri umani abbiano definito delle regole per tali linguaggi, non li hanno inventati: sono nati e si sono evoluti naturalmente;
- I linguaggi formali sono stati progettati dalle persone per applicazioni specifiche. Sono costituiti da sequenze (stringhe) di simboli (caratteri) definiti in un alfabeto. Ne sono esempi la notazione matematica, quella musicale, quella usata in chimica.
- Nei computer i linguaggi di programmazione sono usati per esprimere calcoli.
- Un linguaggio di programmazione a basso livello coincide o differisce di poco dal linguaggio macchina, con poca o nessuna astrazione dai dettagli del funzionamento fisico del calcolatore. Un linguaggio di programmazione ad alto livello, viceversa, fornisce una significativa astrazione dai dettagli del funzionamento della macchina.

Tipi di linguaggi di programmazione

- Esistono molti tipi di stili o paradigmi di programmazione, per esempio programmazione **imperativa** e dichiarativa; tra queste la programmazione funzionale, procedurale, **strutturata**, a oggetti, e altre.

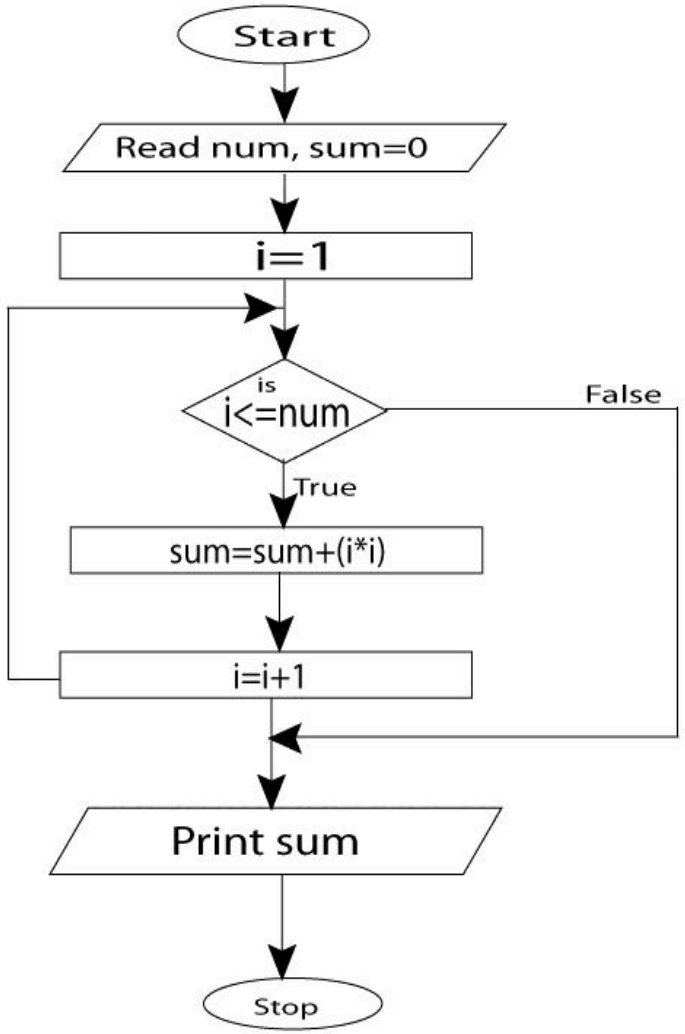
- In questo corso ci occupiamo del paradigma di programmazione imperativa, facente uso di strutture di controllo per costruire funzioni e procedure.



```
N = 7
SOMMA = 0
i = 1

while(i <= N):
        i_quadro = i*i
        SOMMA = SOMMA + i_quadro
        i = i + 1

print SOMMA
```

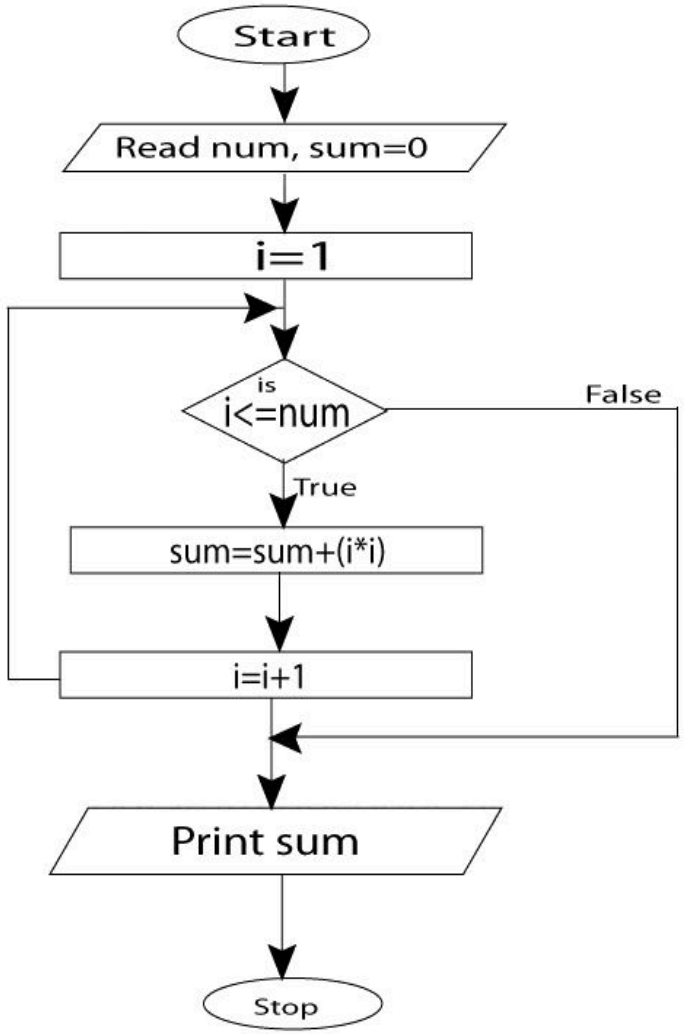


```
N = 7
SOMMA = 0
i = 1

while(i <= N):
        i_quadro = i*i
        SOMMA = SOMMA + i_quadro
        i = i + 1

print SOMMA
```

```
> python somma_quadrati.py
140
```



```c
#include <stdio.h>
#include <stdlib.h>

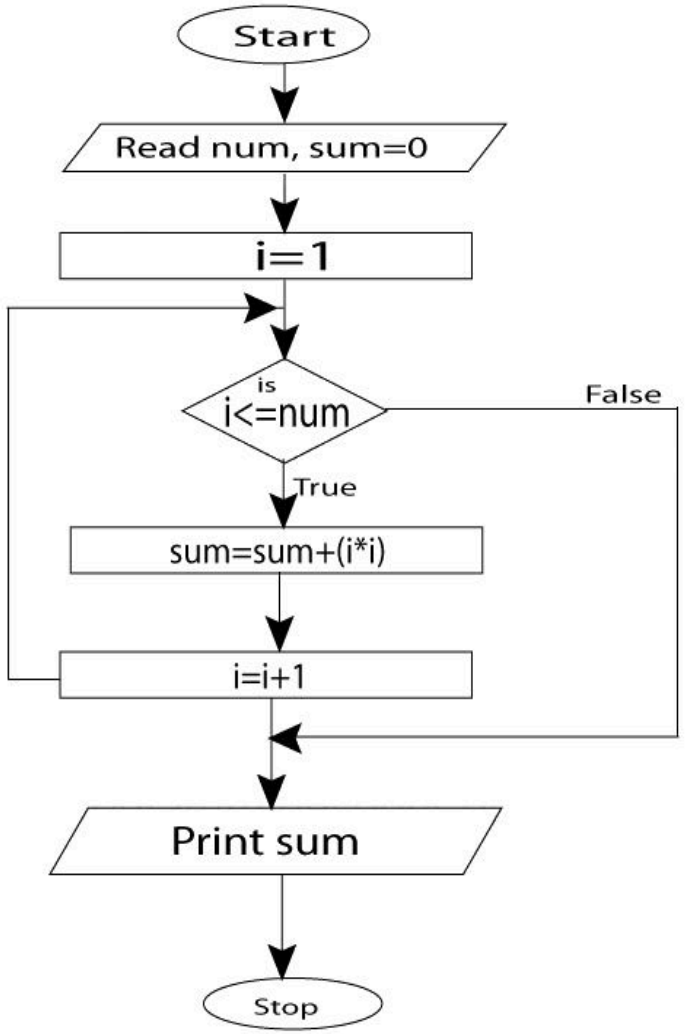
int N = 7;
int i = 0;
int i_quadro;
long int SOMMA = 0;

void main() {

        while (i <= N) {
                i_quadro = i*i;
                SOMMA = SOMMA + i_quadro;
                i += 1;
        }

        printf("%ld\n", SOMMA);

        exit(0);
}
```



```c
#include <stdio.h>
#include <stdlib.h>

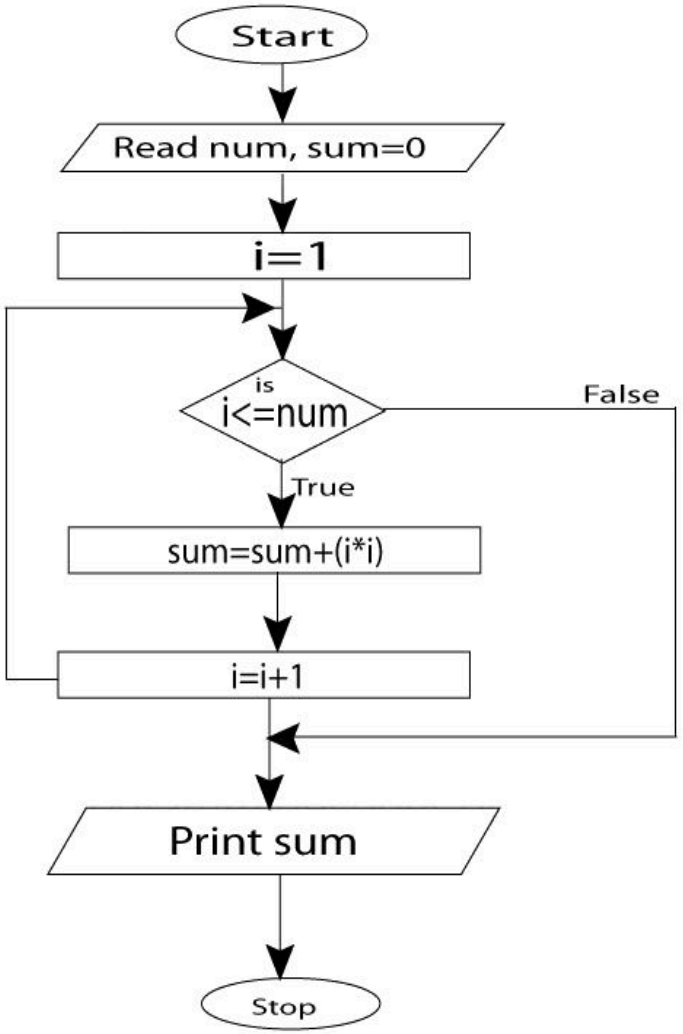
int N = 7;
int i = 0;
int i_quadro;
long int SOMMA = 0;

void main() {

        while (i <= N) {
                i_quadro = i*i;
                SOMMA = SOMMA + i_quadro;
                i += 1;
        }

        printf("%ld\n", SOMMA);

        exit(0);
}
```



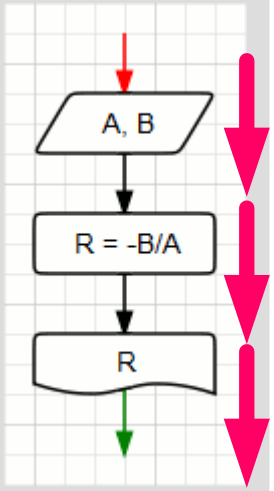
Con **N = 70 000 000**:
Python: 14 secondi - **C:** 0.2 secondi

Programmazione Strutturata

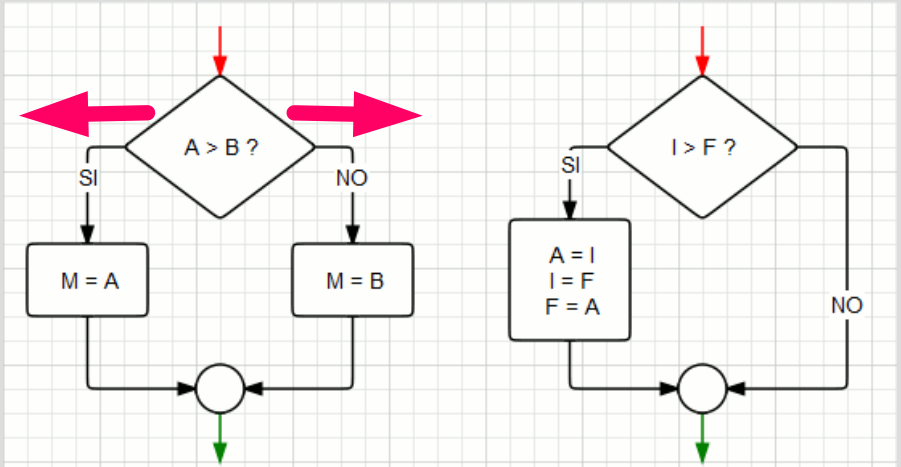
- Teorema di Böhm-Jacopini (1966): qualunque algoritmo può essere tradotto in un programma utilizzando tre sole strutture, dette **strutture di controllo**: la sequenza, la selezione ed il ciclo (iterazione)
- **Sequenza**, Salto, GOTO: possibilità di saltare attraverso le istruzioni
- **Selezione**, cioè la possibilità di decidere in base a certe condizioni quali blocchi di istruzioni eseguire;
- **Ciclo**, cioè la possibilità di eseguire iterazioni sullo stesso blocco di istruizioni;

Strutture di controllo

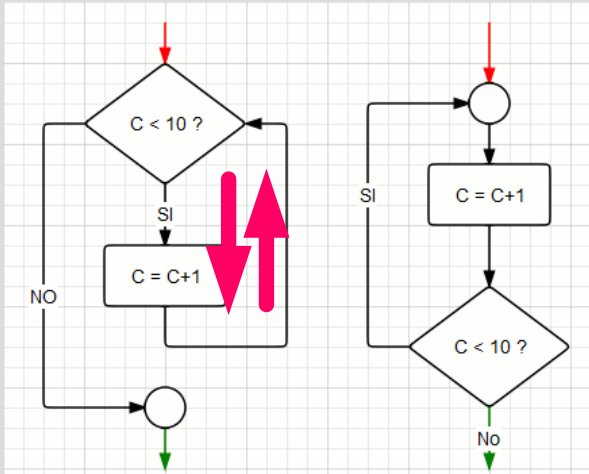
Sequenza



Selezione



Iterazione



Requisiti:
1) **Completezza**: presenza di Sequenza, Selezione e Iterazione;
2) **Singolo punto** di **entrata** e **uscita**;
3) **Componibilità**: sono possibili annidamento e ricorsione.

Componibilità delle strutture di controllo

